Documento de Cobertura del Testing — Sonido Vivo · EP2

Sigue la plantilla `09_Plantilla_Documento_Cobertura_Testing_EP2.docx`.

> **Tecnología.** La pauta impresa de la EP2 nombra Jasmine y Karma; el curso los reemplazó por **Vitest + React Testing Library**, que es lo que usa este proyecto.

---

1. Identificación

Campo	
Sección	DSY1104-005D
Grupo	Proyecto individual (sin número de grupo)
Integrantes	Sol León
Contexto / caso	Forma B — Tienda Sonido Vivo
Repositorio GitHub	https://github.com/s0lci700/SonidoVivo
Versión evaluada	Código de `frontend/` en el commit `896ec76` (último cambio de código). El commit final de la entrega, que solo agrega documentación, se indica en la ficha de identificación
Fecha de medición	20/10/2026

---

2. Configuración del entorno de pruebas

Herramienta	Versión	Para qué se usa
Vitest	4.1	Ejecuta las pruebas; entrega `describe`, `it`, `expect` y `vi` (mocks, spies, relojes falsos)
React Testing Library (`@testing-library/react`)	16.3	Renderiza componentes y los consulta como un usuario (`getByRole`, `getByLabelText`, `findBy…`)
`@testing-library/jest-dom`	7.0	Comprobaciones del DOM: `toBeInTheDocument`, `toBeDisabled`, `toHaveValue`, `toHaveAttribute`
jsdom	29.1	Simula el navegador dentro de Node
`@vitest/coverage-v8`	4.1	Mide la cobertura de código con el motor V8
React Router (`MemoryRouter`)	7	Router en memoria para probar componentes con `Link`, `useNavigate` o `useParams`

Configuración (`frontend/vite.config.js`, bloque `test`):

- `environment: 'jsdom'`: las pruebas corren con un DOM simulado.
- `setupFiles: './src/test/setup.js'`: antes de cada archivo importa `@testing-library/jest-dom/vitest`, limpia `localStorage` antes de cada prueba (cada una parte del catálogo original y sin sesión) y desmonta lo renderizado después (`cleanup`).
- `coverage`: proveedor `v8`, reportes `text` y `html`; incluye `src/**/*.{js,jsx}` y excluye `src/main.jsx` y `src/test/**`.

Comandos (en `frontend/`):

Comando	Qué hace
`npm test`	Ejecuta todas las pruebas una vez (`vitest run`)
`npm run test:watch`	Vuelve a ejecutar al guardar un archivo
`npm run test:coverage`	Ejecuta las pruebas y genera el reporte en `frontend/coverage/` (`index.html`)

Las pruebas viven junto al archivo que prueban, con sufijo `.test.js` o `.test.jsx`.

---

3. Registro de pruebas

Resultado global: 78 pruebas en 15 archivos, 78 OK, 0 errores.

Las 10 pruebas representativas

N°	Archivo / componente	Comportamiento verificado	Tipo	Resultado
1	`utils/carrito.test.js`	Una guitarra con stock 2 agregada tres veces queda con cantidad 2 (`agregarProducto` respeta el stock)	Lógica	OK
2	`utils/carrito.test.js`	`calcularTotal` suma precio × cantidad de todas las líneas	Lógica	OK
3	`utils/validaciones.test.js`	`validarCorreo("docente.duoc.cl")` devuelve `false`	Lógica / validación	OK
4	`services/productoService.test.js`	`crearProducto` con un código existente (`GA001`) lanza «Ya existe…»	Servicio asíncrono	OK
5	`services/authService.test.js`	`iniciarSesion` guarda `{ correo, nombre, rol }` en `localStorage` con la clave `usuarioActual`, sin la contraseña	Spy (`vi.spyOn`)	OK
6	`components/Item.test.jsx`	El clic en Agregar llama una vez a `onAgregar` con `{ codigo, nombre, precio, stock }`	Componente + mock (`vi.fn`)	OK

N°	Archivo / componente	Comportamiento verificado	Tipo	Resultado
7	`components/Item.test.jsx`	El botón muestra «Agregado» y vuelve a «Agregar» a los 1,5 s	Componente + relojes falsos	OK
8	`pages/Login.test.jsx`	Con el servicio de login simulado devolviendo `false`, se muestra «Credenciales incorrectas»	Interacción + mock de módulo (`vi.mock`)	OK
9	`pages/Checkout.test.jsx`	Pagar con el formulario vacío muestra los errores y no vacía el carrito	Formulario + mock (`vi.fn`)	OK
10	`pages/AdminProductos.test.jsx`	Confirmar «Eliminar» quita GA001 de la tabla	Asíncrono + spy (`window.confirm`)	OK

Todas las pruebas, por archivo

Archivo	N°	Qué verifica	Resultado
`utils/carrito.test.js`	13	Tope de stock y mínimo 1 (`limitarCantidad`), agregar y sumar, quitar, cambiar cantidad, total y carrito vacío	13 OK
`utils/validaciones.test.js`	15	`validarTexto`, `validarCorreo`, `validarPassword`, `validarPrecio`, `validarStock`: casos válidos e inválidos	15 OK
`utils/estadoStock.test.js`	3	Stock 0, 3 y 8 → «Agotado», «Últimas 3 unidades», «En stock»	3 OK
`utils/formatearPrecio.test.js`	4	Miles, millones, menos de mil y cero en CLP	4 OK
`services/productoService.test.js`	9	CRUD: crear, rechazar repetido, leer, `null` si no existe, actualizar solo lo enviado, eliminar, rechazar inexistentes, persistencia	9 OK
`services/authService.test.js`	1	Spy sobre `localStorage.setItem`	1 OK
`components/Item.test.jsx`	6	Datos y precio, renderizado condicional del botón, stock 0, mock de `onAgregar`, enlace a la ficha, aviso de 1,5 s	6 OK
`pages/Login.test.jsx`	3	Campo controlado, sin error inicial, mock del servicio	3 OK
`pages/Contacto.test.jsx`	3	Renderiza, errores al enviar vacío, envío válido limpia el formulario	3 OK
`pages/Carrito.test.jsx`	3	El botón + pide la cantidad siguiente, carrito vacío, total	3 OK
`pages/Checkout.test.jsx`	3	Resumen y total, validaciones, retiro en tienda válido vacía el carrito	3 OK
`pages/DetalleProducto.test.jsx`	3	Producto de la URL, código inexistente, campos enviados al carrito	3 OK
`pages/AdminProductos.test.jsx`	4	Lista, Editar enlaza al formulario, Eliminar, cancelar no borra	4 OK
`pages/AdminProductoForm.test.jsx`	5	Precarga al editar, código bloqueado, 8 errores, guarda el precio y vuelve a la lista, código repetido	5 OK
`pages/Admin.test.jsx`	3	Saludo de la sesión, agotado en «Por reponer», Cerrar sesión	3 OK

4. Mocks y spies utilizados

Herramienta	Dónde	Qué reemplaza u observa	Por qué
`vi.fn()`	`Item`, `Carrito`, `Checkout`, `DetalleProducto`	Las funciones que el componente recibe por props (`onAgregar`, `onCambiarCantidad`, `onVaciar`)	El componente no decide qué pasa con el carrito: solo avisa a su padre (`App`). Se prueba el aviso, no el carrito
`vi.mock("../services/authService")`	`Login.test.jsx`	El módulo de sesión completo	Aislar el formulario de `usuarios.json` y de `localStorage`, y forzar el caso de error
`vi.spyOn(Storage.prototype, "setItem")`	`authService.test.js`	Observa `localStorage.setItem` real	Comprobar qué se guarda sin leer `localStorage` a mano
`vi.spyOn(window, "confirm")`	`AdminProductos.test.jsx`	El diálogo nativo de confirmación	jsdom no tiene un `confirm` interactivo; el spy decide la respuesta
`vi.useFakeTimers()`	`Item.test.jsx`	`setTimeout`	No esperar 1,5 s reales

5. Resultado de cobertura

`npm run test:coverage` sobre la versión evaluada, fila `All files`:

Métrica	Resultado	Interpretación
Statements	**75,06 %** (277/369)	Tres de cada cuatro instrucciones se ejecutan durante las pruebas
Branches	**75,78 %** (169/223)	La mayoría de los `if`/ternarios se prueban por ambos lados: dato válido e inválido, con y sin stock, crear y editar

Métrica	Resultado	Interpretación
Functions	**63,75 %** (95/149)	Más baja porque las páginas de presentación sin pruebas (Inicio, Nosotros, Header, Footer) tienen muchas funciones pequeñas de renderizado
Lines	**74,77 %** (249/333)	Coincide con las sentencias

Por carpeta

Carpeta	Líneas	Comentario
`src/utils`	**100 %**	Toda la lógica pura: carrito, validaciones, estado de stock, precio, imágenes
`src/services`	**92,5 %**	CRUD de productos 96,6 %; sesión 80 %
`src/pages`	**80,7 %**	Páginas con lógica entre 80 y 100 %; páginas de presentación en 0 %
`src/components`	**43,6 %**	Item, Boton, Campo, Selector y ResumenPedido al 100 %; Header, Footer, Chip, ItemList y RutaProtegida sin pruebas
`App.jsx`, `layouts/`	0 %	Solo arman rutas y layout; se ejercitan indirectamente al usar la app

Por archivo

Archivo
`utils/carrito.js`
`utils/validaciones.js`
`utils/estadoStock.js`, `formatearPrecio.js`, `imagenProducto.js`
`services/productoService.js`
`services/authService.js`
`services/regionService.js`
`components/Item.jsx`, `Boton.jsx`, `Campo.jsx`, `Selector.jsx`, `ResumenPedido.jsx`
`pages/AdminProductoForm.jsx`, `Contacto.jsx`, `DetalleProducto.jsx`
`pages/Admin.jsx`
`pages/AdminProductos.jsx`
`pages/Checkout.jsx`
`pages/Login.jsx`
`pages/Carrito.jsx`
`App.jsx`, `Header.jsx`, `Footer.jsx`, `Chip.jsx`, `ItemList.jsx`, `RutaProtegida.jsx`, `LayoutPrincipal.jsx`, `Catalogo`

Los archivos de `data/*.json` aparecen en el reporte con 0 % porque son datos, no código; no afectan el análisis.

El reporte HTML navegable queda en `frontend/coverage/index.html` al ejecutar `npm run test:coverage` (la carpeta no se sube al repositorio).

---

6. Análisis

¿Qué componentes o funciones quedaron mejor cubiertos? La lógica de negocio: `utils/` completo (100 %) y el CRUD de `productoService` (96,6 % de líneas). Son las partes donde un error cuesta más (cantidades por sobre el stock, productos duplicados, precios inválidos) y, al ser funciones puras o servicios sin interfaz, se prueban rápido y en aislamiento. Entre los componentes, `Item` (6 pruebas: renderizado, props, renderizado condicional, mock y temporizador) y los formularios de administración, Checkout y Contacto.

¿Qué funcionalidades presentan menor cobertura y por qué?

- Páginas de presentación sin lógica propia (Inicio, Nosotros, NoEncontrado, Footer): solo muestran contenido fijo.
- `Header` (menú móvil, contador, sesión), `Catalogo` (chips de filtro) y `RutaProtegida`: tienen comportamiento, pero se priorizó la lógica y los formularios. Son la siguiente prioridad.
- `ResultadoCompra`: depende del `state` de la navegación que envía el Checkout.
- `App.jsx` y `LayoutPrincipal`: solo declaran rutas y estructura.
- En `Checkout` quedan sin probar la navegación con pago rechazado (línea 81), la vista de carrito vacío (línea 89) y la lista de comunas y la casilla de pago rechazado (líneas 176–198); en `Login`, el mensaje de campos vacíos y la navegación al panel (líneas 23–24 y 30).

¿Qué error o comportamiento inesperado fue detectado gracias a una prueba?

- En `productoService.test.js`, comparar `obtenerProductoPorCodigo("XX001")` sin `await` entregaba una promesa en lugar del producto. La prueba falló y obligó a esperar el resultado.
- En `AdminProductoForm.test.jsx`, la prueba de código repetido se escribió primero sobre la ruta de edición: el formulario guardaba y navegaba. Eso dejó claro que el error de código repetido solo ocurre al **crear**, y la prueba se corrigió para usar `/admin/productos/nuevo`.
- Al probar el formulario a mano se detectó que un stock vacío se guardaba como 0 (`Number("") === 0`) y se agregó la validación de campo vacío.
- En `carrito.test.js`, las pruebas de tope de stock fijaron la regla de `limitarCantidad` (mínimo 1, máximo el stock) antes de conectar el carrito a la interfaz.

¿Qué prueba adicional sería conveniente implementar en una siguiente versión?

1. `RutaProtegida`: sin sesión redirige a `/login`; con sesión muestra la página.
2. `Header`: el botón hamburguesa abre y cierra el menú (`aria-expanded`) y, con sesión, aparecen «Panel» y «Salir».
3. `Catalogo`: el clic en un chip cambia `?categoria=` y filtra la grilla.
4. `Checkout`: despacho con región y comuna, y el pago rechazado que navega a `/compra/error` conservando el carrito.
5. `ResultadoCompra`: con y sin pedido en el `state`.

---

7. Conclusiones

Las 78 pruebas cubren las reglas que sostienen la tienda: el carrito nunca supera el stock, los formularios no aceptan datos inválidos, el CRUD rechaza códigos repetidos o inexistentes y la sesión se guarda sin la contraseña. Separar la lógica en funciones puras (`utils/`) y en servicios (`services/`) permitió probarla al 100 % sin depender de la interfaz, y los mocks y spies aislaron cada componente de sus dependencias (servicios, `localStorage`, diálogos y tiempo). La cobertura total de 75 % se concentra donde importa; lo que queda sin probar es sobre todo presentación, con una lista concreta de pruebas para la siguiente versión. Las pruebas detectaron errores reales durante el desarrollo, y permiten cambiar `services/` por la API del Parcial 3 con una red de seguridad: si el contrato de las funciones se rompe, las pruebas lo muestran.